

MINI ASSESSMENT — PROMPT WORKFLOW DESIGN

Assignment: Transforming Raw Product Notes into a Polished Blog Post

STUDENT	GOWTHAM KAKKE	REGISTER NO.	99230040028
SUBJECT / TASK	Prompt Workflow Design	WORKFLOW TYPE	Sequential Multi-Step Prompting

1. CONCEPT

Purpose: Convert disorganized meeting notes about a new “Smart Analytics” dashboard into a customer-facing blog post through a controlled prompt chain. The workflow separates planning, drafting, and quality review so that each stage depends on the output of the previous stage.

Problem: Writing directly from messy notes can lead to missing benefits, unclear structure, inconsistent tone, and failure to meet formatting constraints. A multi-step workflow creates a repeatable process with an explicit evaluation gate.

2. OBJECTIVE

Design a sequential prompt workflow that extracts the three main user benefits, creates a structured outline, generates a constrained first draft, and then performs a mandatory self-review before producing the final corrected blog post.

3. WORKFLOW OVERVIEW

1. PLAN	2. DRAFT	3. CRITIQUE	4. REFINE
Extract benefits + outline	Write from Step 1 output	Check constraints + CTA	Correct failures + finalize

4. WORKFLOW STEPS

Step 1 — Structuring & Planning (The Setup)

Goal: Establish the persona, feed the initial raw data, extract the three strongest user benefits, and organize the information into a structured outline.

Prompt 1: Act as an expert content marketer. I am going to give you raw, messy notes " "from a product meeting about our new “Smart Analytics” dashboard. Your task is to extract " "the 3 main user benefits from these notes and create a structured outline for a 500-word " "customer-facing blog post. Organize the outline with clear headings and bullet points. " "[Insert Messy Raw Notes Here]

Expected output: Expected output: A structured outline containing the top 3 user benefits and the proposed article structure.

Step 2 — Drafting (The Execution)

Goal: Generate the first draft strictly from the outline produced in Step 1.

Prompt 2: Excellent. Now, using only the outline you just generated, write the first " "draft of the blog post. Constraints: keep the tone enthusiastic but professional; use short " "paragraphs with a maximum of 3 sentences each; do not exceed 500 words; include a placeholder " "for an image; and preserve the three main benefits from the outline.

Expected output: Expected output: A complete first draft that follows the structure and all stated constraints.

Step 3 — Critique & Refinement (The Required Gate)

Goal: Act as a strict senior editor and evaluate the draft against explicit acceptance criteria before finalization.

Prompt 3: Before we finalize this, act as a strict senior editor. Review the draft " "against this checklist: (1) Is it within the 500-word limit? (2) Is there a clear, persuasive " "Call to Action at the very end? (3) Does every paragraph contain no more than 3 sentences? " "First, list your critique and identify every failed criterion. Second, output the final corrected " "version that fixes those errors without changing the approved core benefits.

Expected output: Expected output: A concise bulleted self-critique immediately followed by a corrected final blog post.

Step 4 — Final Validation

Goal: Verify that the refined article is ready for submission and that the required constraints remain satisfied.

Validation instruction: Check the final article for the three approved user benefits, " "professional and enthusiastic tone, maximum 500 words, paragraph-length compliance, image placeholder, " "and a final CTA. If any requirement is still missing, revise only the affected part and re-check.

Expected output: Expected output: A validated final article with all required constraints satisfied.

5. PROMPT CHAINING ARCHITECTURE

The workflow is dependency-based: Step 1 creates the outline; Step 2 uses only that outline; Step 3 evaluates the resulting draft against fixed criteria; and Step 4 validates the corrected version. This preserves the progression and mandatory evaluation gate described in the reference material.

STAGE	INPUT / CONTEXT	OUTPUT
1	Raw product meeting notes	3 user benefits + structured outline
2	Step 1 approved outline	First draft of blog post
3	Step 2 draft + checklist	Critique + corrected final draft
4	Step 3 final draft + validation rules	Validated submission-ready article

6. DEMONSTRATION INPUT

Scenario: The input consists of disorganized meeting notes about a new “Smart Analytics” dashboard. The notes are supplied to Step 1 as raw material; the model must extract the three main customer benefits before any blog drafting begins.

Important workflow rule: The raw notes are used only as the initial input. Later stages should use the outputs generated by earlier stages rather than independently inventing a new structure.

7. EXPECTED FINAL WORKFLOW RESULT

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Planning	3 benefits identified + structured outline
Phase 2	Drafting	Professional first draft $\leq$ 500 words
Phase 3	Evaluation Gate	Checklist-based critique completed
Phase 4	Refinement	Corrected final article + CTA
Phase 5	Validation	All constraints confirmed before submission

8. KEY BENEFITS OF THE WORKFLOW

- Separates planning from execution, reducing ambiguity in the drafting stage.
- Keeps the generated article tied to the approved outline and user benefits.
- Uses explicit constraints such as word count, paragraph length, tone, image placeholder, and CTA.
- Introduces a mandatory evaluation gate before finalization.
- Creates a repeatable workflow that can be reused for similar content-generation tasks.

9. DISCUSSION QUESTIONS & ANSWERS

1. What is the advantage of using variables or placeholders instead of hard-coding input?

Templating separates the fixed prompt structure from changing information. This allows the same workflow to process different sets of notes without rewriting the entire prompt each time.

2. When would `.format()` be enough?

Python’s `.format()` is sufficient for simple, linear substitutions, such as inserting a crop/product name or a short variable into a static prompt. It is appropriate when no complex conditional or looping logic is

required.

3. When might Jinja2 be useful?

Jinja2 becomes useful when prompt structure must change conditionally or when repeated content must be generated. For example, conditional blocks can include different instructions based on a supplied condition, while loops can process multiple items.

4. Why should prompts be versioned?

Prompt versioning provides an audit trail when instructions or model behavior change. It also makes it possible to compare versions and roll back a problematic prompt.

5. What is a prompt regression?

A prompt regression occurs when a change intended to improve one behavior unintentionally reduces the model's performance on an existing task or input type.

6. Why use the same evaluation set when comparing prompt versions?

A fixed evaluation set provides an apples-to-apples comparison. If both the prompt and test examples change, it becomes difficult to determine whether a performance difference came from the prompt or from the changed inputs.

10. CONCLUSION

This assignment implements a structured, sequential prompt workflow in which each stage has a defined goal, input, and expected output. The required critique gate ensures that the final blog post is checked against the specified constraints before it is treated as complete.